SQL Query Library — Design Doc

Project: Loan Default Prediction (HMEQ dataset) **Artifact:** SQL query library (~19 queries) **Positioning:** Credit middle office / operations analyst **Status:** Inventory locked. Queries to be built next.

---

1. Scope & Sourcing Decision

Primary table: raw `hmeq.csv`. Plus `predictions.csv` for the monitoring section. Do **not** load a pre-cleaned CSV.

Why raw: The Power BI dashboard already runs on cleaned data and owns the *presentation* layer. The SQL library earns a distinct role only if it runs against raw data — it becomes the layer that *catches and transforms* problems as data lands, which is exactly the middle-office analyst's job. The 203% DTI record only exists to be found if the queries hit raw data.

Clean data is produced by SQL, not imported. One query — a CTE or `CREATE VIEW hmeq_clean` — reproduces the Python cleaning steps in SQL (IQR capping, median/mode imputation, banding). The clean table becomes a SQL artifact you can point to, not a file you loaded. This is the "same findings, second language" story.

Documented limitation (KNN): The notebook uses KNN imputation for DEBTINC, which is a model, not a SQL operation. The SQL clean layer reproduces the *deterministic* cleaning only — capping, banding, and median imputation for DEBTINC as a **stated simplification** — and a header comment notes that the notebook remains the source of truth for the KNN-imputed training set. Naming this limitation is a maturity signal, not a gap.

---

2. Question Inventory

Every query carries its business question as a comment header. Grouped into five sections following a real analyst's workflow: **profile → summarize → segment → engineer → monitor.**

Section 1 — Data Profiling & Quality Controls

1. **Is every loan record unique?** — Row count + distinct `Record_ID`, flag duplicates.
2. **Where are the gaps, and how bad?** — Missing-value count and % per column (surfaces the ~21% DEBTINC hole, DELINQ/DEROG gaps).
3. **Which records carry impossible values?** ⭐ — DTI > 100 with full rows; finds the five famous records incl. the 203%. *Signature query.*
4. **Are the category codes clean?** — Distinct `JOB` / `REASON` vs. expected list; catches

typos and new codes.

5. **Do any loans defy basic loan logic?** *(optional)* — `MORTDUE > VALUE`, or zero/negative `LOAN`; collateral-vs-debt sanity check.

Section 2 — Portfolio KPIs

6. **What's the headline health of the book?** — One query: total loans, default rate, total exposure, avg loan size. (Page 1 KPI row.)
7. **Does default rate differ by loan purpose?** — Default rate + exposure by `REASON`.
8. **How is the book distributed by loan size?** — `CASE`-based `LOAN` bins with default rate per band.

Section 3 — Risk Segmentation *(heart of the library)*

9. **Which occupations default most?** — Default rate by `JOB`, sorted (the 35%-vs-13% finding).
10. **Does delinquency history predict default?** — `DELINQ` bands via `CASE`, default rate per band.
11. **At what debt load does risk accelerate?** — `DTI` bands, default rate per band.
12. **Does thin credit history mean higher risk?** — `CLAGE` bands, default rate per band.
13. **Does risk compound?** ⭐ — `JOB` × `DELINQ` band multi-dimension cut. Goes past what the dashboard shows. *Signature query.*

Section 4 — Advanced Patterns *(technical depth)*

14. **Reproduce the cleaning pipeline in SQL.** ⭐ — CTE or `CREATE VIEW hmeq_clean`: cap → impute → band. KNN caveat documented here. *Signature query.*
15. **Rank and slice with window functions.** — Rank JOB segments by default rate, or `NTILE` the book into exposure deciles with default rate per decile.
16. **What are the top-N riskiest segments overall?** — Self-contained CTE + ranking to surface the 5 worst borrower profiles.

Section 5 — Model Monitoring *(the section nobody else has)*

17. **Reconstruct the confusion matrix.** — `SUM(CASE WHEN...)` from `predictions` → 165/73/103/851. Third reconciliation of the same numbers.
18. **Compute recall and precision from scratch** — In SQL, off the matrix above.
19. **Who did the model miss?** ⭐ — JOIN `predictions` to `hmeq` on `Record_ID`, profile the false negatives; find segments where the model is blind. *Signature query.*

Total: 19 questions (17 if #5 is dropped and #15/#16 merged).

Protect these four if trimming: #3, #13, #14, #19 — the queries an interviewer remembers.

---

3. Connective Narrative (drop into README)

Four artifacts, one throughline:

- **Python notebook** — the analytical engine. Finds the drivers (DELINQ, CLAGE, DEROG, DEBTINC), builds and tunes the model.
- **Excel control workbook** — the manual, analyst-facing control layer (six-tab governance view).
- **SQL library** — the database-native, reproducible version of profiling + segmentation + monitoring. The queries an ops analyst keeps in a drawer.
- **Power BI dashboard** — the executive presentation layer.

Two reconciliation stories (both genuine talking points):

- **Banded logic in three languages:** Power Query M → DAX `SWITCH(TRUE())` → SQL `CASE`. Same DELINQ / DTI / CLAGE bands, three engines.
- **Confusion matrix reconciled three ways:** 165 / 73 / 103 / 851 computed independently in Python → Power BI → SQL.

Workflow arc: profile → summarize → segment → engineer → monitor — the shape of a real analyst's process, and the shape of this library.

---

4. Open Decisions (settle before or early tomorrow)

- **Engine:** SQLite (zero setup, runs in the repo, portable) vs. Postgres/Docker (closer to job reality). Lean: SQLite for portability.
- **Deliverable format:** one annotated `.sql` file vs. a Jupyter notebook with `pandas.read_sql` showing results inline. The notebook version demos better.